Multi-Level Parking Lot Management System

(Efficient automated system for multi-level parking management)

Data Structures PBL Proposal

(Group - 12)

Submitted To:

Ma'am Daniya Jadoon

Department of Software Engineering

Batch 2024
Semester – Fall 25

Dec 5, 2025

1. Project Title

"MULTI-LEVEL PARKING LOT MANAGEMENT SYSTEM"

2. Group Members

- Muhammad Hassan (F24608010)
- Muhammad Hassan Khawaja (F24608030)
- M Rehan Muddassir Ranjha (F24608041)

3. Problem Statement

a. What Problem Will the System Solve?

Manual parking management is slow, error-prone, and inefficient. Without automation, slot assignment, vehicle lookup, and parking history tracking become difficult, causing delays and mismanagement.

This system automates slot allocation, vehicle tracking, and record management to ensure quick search, accurate availability updates, and organized storage of parking data.

b. Why the System Is Needed

- Manual logs cause incorrect slot assignment and missing records.
- Finding a parked vehicle is slow.
- Real-time occupancy across multiple levels is difficult to maintain.
- Parking history and fee data are rarely stored properly.

The system provides a reliable, scalable, and faster digital alternative suitable for malls, offices, and public parking areas.

4. Project Objectives

This project aims to implement a multi-level parking management system using custom C++ data structures without STL, automating vehicle check-in, check-out, and slot assignment while supporting fast vehicle lookup through efficient searching.

It ensures persistent data storage using file handling, follows a modular and maintainable architecture, and provides a clean, user-friendly, color-coded CLI interface. The system is designed to demonstrate strong understanding of pointers, dynamic memory management, and algorithmic problem-solving.

5. Features / Functional Requirements

Core Features

- Automatic slot allocation on vehicle check-in.
- Vehicle check-out with fee calculation.
- Multi-level slot representation.
- Real-time free/occupied slot display.
- Search vehicle by registration number.
- Waiting queue when parking is full.
- Parking history (time-in, time-out, charges).
- Save/load system state using files.
- ASCII map visualization.
- Input validation + error handling.
- Color-coded CLI (green for available, red for occupied).

6. Data Structures to Be Used

Data Structure	Purpose
Linked List	Dynamic multi-level parking slot storage and traversal.
Queue	Manages waiting vehicles in FIFO queue system.
Hash Table	Provides instant vehicle lookup using custom hash mapping.
Stack	Used for history or undo operations (LIFO behavior).
BST	Used for sorted parking history or fee records.

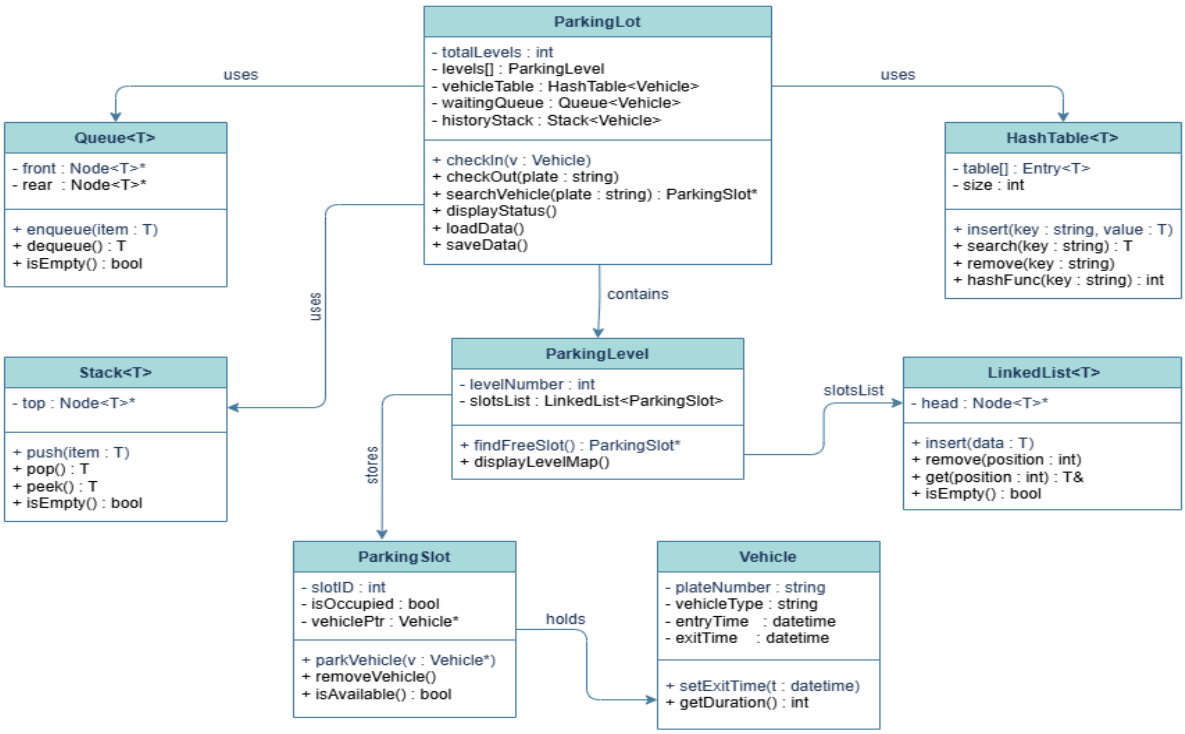
7. Algorithms to Be Used

Algorithm	Purpose
Hash Table Lookup	Fast search with linear probing/chaining.
In-order Traversal	Generates sorted parking history.
Slot Allocation	Sequential search through linked lists for nearest slot.
Fee Calculation	Computes parking charges based on time duration.
Queue Processing	Automatically assigns slots to waiting vehicles

8. Proposed Menu Structure

```
-------------------------------------------
    Multi-Level Parking Lot System
-------------------------------------------
1. Vehicle Check-In
2. Vehicle Check-Out
3. Search Vehicle by Number
4. Display Parking Status (ASCII Map)
5. View Parking History
6. Show Waiting Queue
7. Save & Exit
-------------------------------------------
```

9. System Architecture Overview (Class Diagram)



10. Project Timeline (10 Days)

Day	Tasks
Day 1	Finalize requirements, system design, UML diagram
Day 2	Implement custom LinkedList structure
Day 3	Implement custom Stack+Queue, begin HashTable structure.
Day 4	Finish custom HashTable; test all DS individually.
Day 5	Implement ParkingSlot, ParkingLevel classes
Day 6	Implement ParkingLot (allocation, check-in/out logic)
Day 7	Implement searching, fee calculation algorithms
Day 8	Implement file handling (save/load system state) and error-checking.
Day 9	Develop full CLI menu, ASCII map, color-coded output
Day 10	Final testing, debugging, documentation, and preparing final report